MobileVC 后端架构现状分析

> 分析日期：2026-05-01

1. 总体架构

后端为单进程 Go HTTP 服务器，通过 WebSocket 与 Flutter 客户端通信。核心职责是代理终端命令执行（Claude CLI / Codex CLI / 任意命令），并管理 AI 会话生命周期。

1.1 入口与路由

入口文件 cmd/server/main.go，启动后注册以下 HTTP 路由：

路由	用途
`/ws`	WebSocket 主通道，所有客户端操作经此
`/healthz`	健康检查
`/version`	版本信息
`/download`	文件下载（需 token）
`/api/tts/synthesize`	TTS 语音合成（可选）
`/api/tts/healthz`	TTS 健康检查（可选）
`/`	内嵌 Web 前端静态资源

1.2 依赖的外部服务

- **APNs (Apple Push Notification service)** — HTTP/2 推送通知，环境变量 APNS_* 控制
- **TTS

---

2. 模块分层

```
┌──────────────────────────────────────────────────────────────────────┐
│ cmd/server    启动入口，组装依赖        │
├──────────────────────────────────────────────────────────────────────┤
│ internal/ws    WebSocket 层，协议路由   │ 约 238KB（handler.go 超大）
│              推送通知辅助          │
├──────────────────────────────────────────────────────────────────────┤
│ internal/runtime 运行时管理，命令调度    │ 约 35KB manager.go
│              权限决策，Session 恢复    │
├──────────────────────┬──────────────────────────────────────────────┤
│ runner    │ session    protocol    │
│ 命令执行器  │ AI 状态机   │ 事件协议    │
│ PTY/Exec   │ 控制器     │ 数据结构    │
├──────────────────────┴──────────────────────────────────────────────┤
│ store    文件持久化层 (JSON 文件)     │
├──────────────────────────────────────────────────────────────────────┤
│ config / push / tts / adb / skills /      │
│ claudesync / codexsync / adapter / logx    │
└──────────────────────────────────────────────────────────────────────┘
```

---

3. 核心模块详解

3.1 `internal/protocol` — 协议定义（约 55KB event.go）

定义了完整的服务端→客户端事件类型（约 30+ 种）和客户端→服务端请求类型。

核心数据结构：

- RuntimeMeta — 横切所有事件的运行时元信息（会话 ID、权限请求 ID、模型、引擎、生命周期状态等），通过 ApplyRuntimeMeta() 注入到每个事件中

- Even

主要事件类型：

类别	事件
流输出	`log`、`progress`、`error`
AI 状态	`agent_state`、`ai_status`、`runtime_phase`
用户交互	`prompt_request`（权限/输入等待）、`interaction_request`
步骤/代码	`step_update`、`file_diff`
会话管理	`session_created`、`session_list_result`、`session_history`、`session_resume_re
文件系统	`fs_list_result`、`fs_read_result`
目录同步	`skill_catalog_result`、`memory_list_result`、`catalog_sync_*`
权限规则	`permission_rule_list_result`、`permission_auto_applied`
ADB 投屏	`adb_devices_result`、`adb_frame`、`adb_webrtc_*`

3.2 `internal/runner` — 命令执行器

定义了运行器接口和执行模式：

- **ModeExec** — 普通命令，通过 ExecRunner 执行，非交互式
- **Mo

核心接口 Runner：

type Runne

```
  Run(ctx context.Context, req ExecRequest, sink EventSink) error
  Write(ctx context.Context, data []byte) error
  Close() error
}
```

扩展接口（类型断言使用）：

- Inter

WinPTY/Windows 支持： runner.go 中包含大量 Windows shell 检测逻辑（Git Bash、PowerShell、cmd、Claude Node.js entry 等）。

Codex App Server 模式： codex_app_server.go（约 32KB）专门处理 Codex 的 stdio JSON-RPC 协议。

3.3 `internal/session` — 会话状态机

Controller 维护单个 AI 会话的状态机：

状态流转：

IDLE → THIN

关键功能：

- 推断

3.4 `internal/runtime` — 运行时管理

核心类型：

- man

主要能力：

- Exec

管理型 Session ID： 自动为 Claude 命令注入 --session-id <uuid> 以追踪会话。

3.5 `internal/ws` — WebSocket 层

handler.go（约 238KB，极大）： 这是整个后端最庞大的文件，包含：

- Web

runtime_sessions.go： 运行时会话注册表，管理 sessionID → runtime session 的映射，支持：

- 监听

push_helper.go： 推送通知辅助逻辑，含去重防抖（30 秒）、连接状态检查。

permission_decision.go： 权限决策执行逻辑，处理 deny/allow 的不同路径。

permission_rules.go： 权限规则管理（CRUD、匹配检测、自动应用）。

slash_command.go： Slash command 解析和执行。

3.6 `internal/store` — 持久化层

基于 JSON 文件存储：

- **Store 接口：** 定义完整的 CRUD 接口
- **File
  快照

存储内容： 每个 session 一个 JSON 文件，包含 SessionSummary + ProjectionSnapshot（完整的投影状态快照）。

3.7 辅助模块

模块	大小	用途
`internal/adb`	14KB	Android 设备管理、WebRTC 投屏
`internal/adapter`	9KB	日志解析（ANSI 剥离、通用解析器）
`internal/claudesync`	26KB	Claude 会话 JSONL 同步
`internal/codexsync`	17KB	Codex 会话 JSON/JSONL 同步
`internal/config`	5KB	配置加载（环境变量 + YAML）
`internal/logx`	1KB	结构化日志
`internal/push`	4KB	APNs 推送服务
`internal/skills`	22KB	Skill 加载与注册
`internal/tts`	23KB	ChatTTS HTTP 语音合成

---

4. 数据流

```
Flutter Client
  |
  | WebSocket (JSON 事件)
  ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│ ws.Handler                                                   │
│  ├── 认证 (token)                                            │
│  ├── 事件路由 (action dispatch)                              │
│  ├── 推送通知触发                                            │
│  └── runtime session registry                                │
└──────────────────────────┬───────────────────────────────────┘
                           │
                           ▼
                 ┌──────────────────────┐
                 │ runtime.Service      │
                 │  ├── controller      │ (状态机)
                 │  ├── manager         │ (Runner 生命周期)
                 │  └── Execute / SendInput / ...
                 └──────────────────────┘
                           │
                           ▼
                 ┌──────────────────────┐
                 │ runner.Runner        │
                 │  ├── PtyRunner       │ (Claude / Codex PTY)
                 │  └── ExecRunner      │ (普通命令)
                 └──────────────────────┘
                           │
                           ▼
                 ┌──────────────────────┐
                 │ PTY subprocess       │
                 │ claude / codex       │
                 └──────────────────────┘
```

输出流返回路径：Runner EventSink → runtime.Service（注入 meta）→ session.Controller.OnRunnerEvent()（状态变更）→ handler（广播/持久化/推送）

---

5. 关键设计模式

5.1 事件注入链

每个 Runner 产出的事件经过 protocol.ApplyRuntimeMeta() 注入当前的 RuntimeMeta（session ID、模型、权限模式等），确保 Flutter 端能获取完整的上下文信息。

5.2 多连接共享

同一个 session 可以有多个 WebSocket 连接（例如手机 + 手表），通过 listener 模式广播事件。17 分钟无连接后自动清理。

5.3 断线重连恢复

通过 eventCursor 机制，客户端重连时可以指定已收到的最后一个 cursor，服务端补发之后的所有 pending events。

5.4 自动 Session Recovery

当向空闲的 PTY Runner 发输入时，SendInputOrResume 会自动通过 --resume 重建 Claude 进程并转发输入。

5.5 权限流

1. Claude 通过 stdio 发送权限请求 JSON
2. Pty

---

6. 当前问题与风险

6.1 handler.go 过大（238KB）

internal/ws/handler.go 单一文件已膨胀到约 238KB，包含过多的职责。建议按功能拆分为多个文件（如 session_handlers.go、execution_handlers.go、catalog_handlers.go 等）。

6.2 测试文件耦合

handler_test.go（约 207KB）同样巨大。部分测试试图"兼容新后端"导致测试代码复杂化，需要重新审视测试策略。

6.3 文件存储方案

file_store.go（33KB）使用文件系统存储 JSON，并发写入时有 race 风险。如果 session 数量增长，可能成为瓶颈。建议考虑 SQLite 替代。

6.4 Windows 代码耦合

runner.go 中包含大量 Windows 特定逻辑（shell 检测、WinPTY 等），在 macOS/Linux 部署时是死代码但增加了维护负担。建议用 build tags 分离。

---

7. 依赖项

github.com/gorilla/websocket	— WebSocket
github.com/creack/pty	— PTY
golang.org/x/net	— HTTP/2 (APNs)
golang.org/x/crypto	— SSH (端口映射)

无外部数据库依赖，无消息队列，架构简洁但扩展性有限。